

TITLE: Introduction to Data cleaning, transformation and
Visualization

Participant Name: Muizz Ahmed

Roll Number: 240911088

Domain: Software & ML

Repository Link: https://github.com/Muizzahmed786/Synergy_TP

Submission Date: 28/06/2026

1) Abstract

The current technical report presents a conceptual overview of the data engineering pipeline implemented for loading, cleaning, and visualizing student dataset.

The aim of this task is to turn unstructured information into a clean and structured form. Git is used for the project to facilitate a clean version control, and all operations are performed within the virtual environments. The initial task of the project features a clear comparison between the manual file parsing and vectorized manipulation of data via Pandas. The parsed files are cleaned by removing duplicates, correcting the category text, extracting numeric values from the units, and addressing missing values. Validation steps to ensure that constraints are implemented, and Matplotlib is utilized to generate visual charts to visualize the tabular data.

2) Introduction

Whenever we're working on data, it is never the case where we are assigned with a clean and complete data. As such, when collecting data from a number of sources, there are usually mistakes in the raw data files. These consist of missing text fields, duplicated data, poor categorizations and unit inconsistencies. Trying to analyse this raw and poorly managed data will cause programs to fail this problem requires a process where we treat the raw data before analysing it to ensure accuracy and reliability of data.

Project setup:

Setup of the project structure via terminal commands as well as the creation of a structured Git repository for task wise maintenance.

Manual Parsing Analysis:

Creating a file reader with no external packages to understand how the string parsing and modification work.

High-Level Extraction:

Using the same file reading logic with the Pandas package to understand how fast and efficient vectorized methods perform.

Data Cleaning: Removing duplicates, isolating the real numbers from the unit text and establishing imputation rules.

Statistical Visualization:

Creating clean programmatic visualizations using Matplotlib.

3) Development Environment and Version Control

3.1) Git and GitHub Tracking

The use of tools such as Git is an essential part of the process of collaboration in software development. Rather than having to rename files manually, Git commits changes to code accordingly. In doing so, it keeps track of all changes that have been made to the code, allowing for easy reverting or comparison, without any loss of data. The use of feature branches separates the main branch code, which allows parallel implementation of features. Thus, the live system will never be affected by any experimental code.

3.2) Virtual Environments and Dependency Isolation

Running within a dedicated Python virtual environment will avoid any potential issues related to system dependency. When installing libraries globally, the update of an outside library may solve one problem but may cause another issue. Therefore, virtual environments provide independent python runtime environments to avoid any system dependencies clashes.

List of Dependencies (requirements.txt): This document contains all necessary libraries with their respective versions required to execute the program. This way, other members of the team will be able to duplicate the same runtime environment setup locally.

Files to Exclude from Version Control (.gitignore): This file is meant for instructing Git to intentionally exclude certain files from being stored in the remote repository.

4) Python and File Handling Concepts

4.1) Basic Programming concepts

Having a basic understanding of basic python concepts is necessary in order to complete more intermediate and advanced tasks. Functions are used to divide the code into small, reusable pieces, each of which has its own purpose, such as reading data or processing a particular column. It ensures that the code is modular in nature and is easily maintainable.

The two main Python objects for storing data in memory are:

Lists: It is an ordered and mutable data structure that contain linear data, such as data rows or filtered student lists.

Dictionaries: It consists of key-value pairs that describe structured records, for example, records with columns "name" and "score."

4.2) File handling, Type Hinting, and Error Handling

Type hinting: By explicitly defining type hints on variables and function parameters (df: pd.DataFrame or None), it clearly defines what does a function accept and what does in return.

File handling: Using file input/output, the files will automatically be closed irrespective of any problems encountered during reading and writing. This makes sure there are no leaks and files are not corrupted.

Exception handling: Raw data may contain corrupted textual values where there should be numerical values. By having a try-except block when performing such operations, any possible exceptions will be handled, e.g., ValueError. In such a case, the program does not break down instead it records the issue and uses a safe default value as provided.

5) CSV Parsing and Pandas

5.1) Understanding the CSV Format

A Comma Separated Value (CSV) file is a plain text file that contains tabular data. Since it is a plain text file, a standard CSV file uses specific components to layout rows and columns:

The Header Row: The very first line of the text file defines the column names (e.g., name, domain, score), which serve as the labels for all subsequent records.

Rows and Records: Each subsequent line in the file represents a single horizontal data record or data row.

Delimiters: A text character most commonly (,) acts as a delimiter to mark the boundary between separate data fields within a single line.

Type Conversions: Since a CSV file is read entirely as text strings, the program must explicitly convert numeric fields (like scores or hours) into integers or floats before performing any mathematical operations.

5.2) Manual Parsing vs. Pandas Structural Vectorization

1.) Manual Parsing:

data loading is done manually via simple Python loops and string manipulation without the use of additional modules. The file is opened, the first line is read to extract headers, and further lines are processed one by one using the standard sequential loop. Individual elements of each line of text string are manually separated using a comma delimiter while ignoring blank lines in order to avoid errors. The text strings are then used together with the header strings to construct dictionaries of rows. Although this method works fine for very small files, it is not suitable for large data sets.

2.) Parsing using Pandas Module

On the other hand, csv parsing using pandas employs the use of the Pandas library that uses the DataFrame object to load the data into memory. Pandas automates the process of detecting the headers, delimiters and guesses the data types automatically. The main benefit offered by Pandas is structural vectorization. Rather than looping through the rows using loops, vectorization performs calculations on the entire data column at once. Internally, Pandas invokes functions written in compiled C code. This approach eliminates the overhead created by using loops and offers much better performance. We need to ensure that the Pandas uses int64, float64 as data types for numeric fields.

6) Data Cleaning and Validation

6.1) The Purpose of Systematic Data Cleaning

Data obtained in their raw form may be dirty and inaccurate. Data should be systematically cleaned before being fed into any analysis process. The presence of missing data, duplicates, and text units that don't match would result in biased results and system errors when calculating statistics such as mean values.

6.2) Cleaning Rules and Decisions

To fix the structural and numeric issues present in the raw file, the program applies explicit cleaning functions in a strict sequential order:

Deleting Duplicate Rows: The program searches for rows that have similar entries and eliminates any duplicates, except for the first instance encountered, which prevents duplication of student numbers due to the same row appearing more than once.

Categorical Value Standardization: In text-based inputs, there may be different naming conventions for categorical values like "ml", "ML" or "MACHINE LEARNING". These values are transformed to lower case, stripped of any trailing whitespaces, and mapped to predefined target categories ML, Web, Electronics, and Mechanical.

Numeric Value Extraction: Phrases in text that denote numbers such as “nine” or “two” are changed into their corresponding numeric values (9 or 2).

Column Normalization: Columns having measurement units of different types, for example, height units given in meters (1.62 m) and centimetres (170 cm), are handled accordingly. The meter units are then multiplied by 100 to convert all the column units to centimetres. Lastly, the column names are updated to reflect the type of units.

Dealing with Missing and Invalid Entries: The missing entries are populated with the median value of that column. In case an empty field for the performance or studying time is found, then it will be replaced with the default value of 0. The out of range invalid entries, for instance, the percentage of attendance less than 0% or greater than 100%, are restricted to the range of [0, 100].

7) Visualization

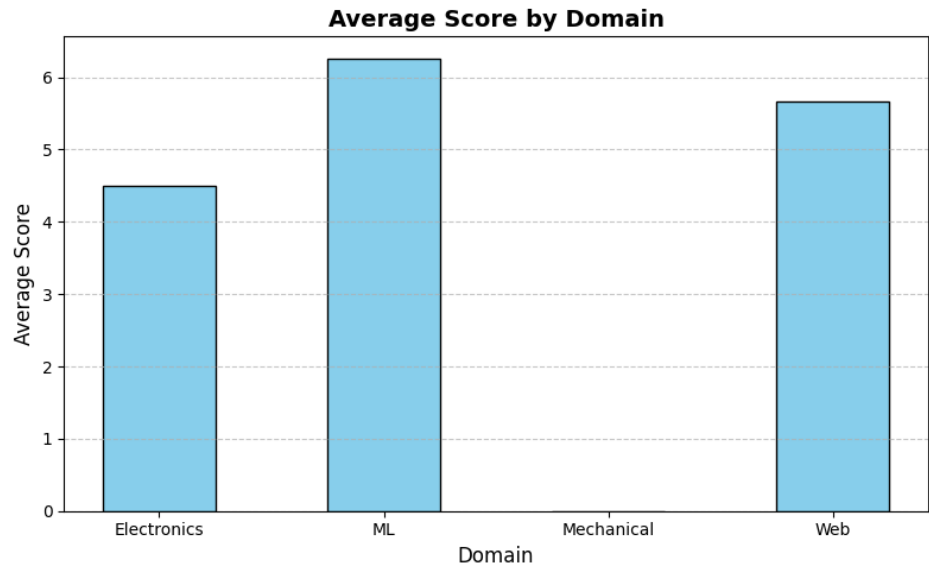
7.1) Purpose of Data Visualization

Cleaned up text data in tables can be hard to understand just from looking at them. There isn't an easy way to find patterns through plain text data. Turning clean data sets into graphs helps visualize numbers in a shape that make understanding much easier.

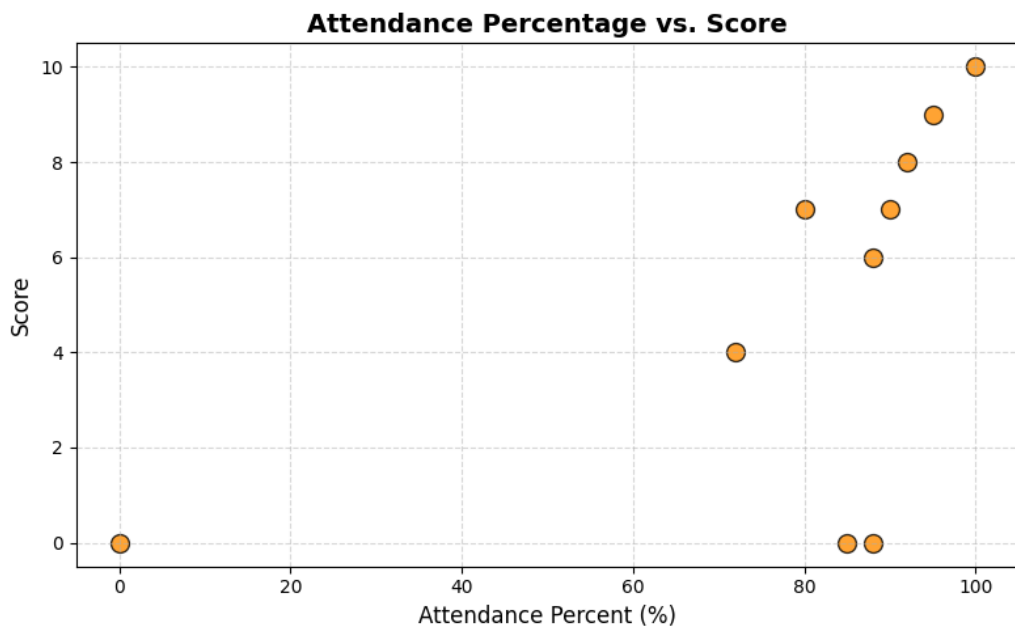
For this task, the program works with Matplotlib and does not utilize any high-level packages such as Seaborn. Every plot has a proper title, axis labels, and y-axis grid. *plt.tight_layout()* is used prior to saving the file to solve the problem with clipping of labels.

7.2) Analysis of generated plots

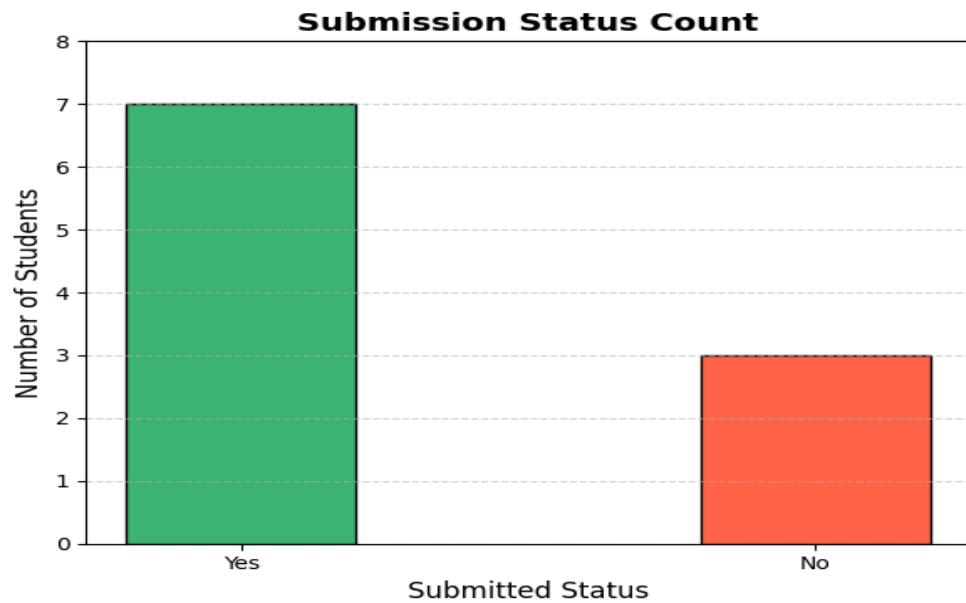
Average Score per Domain: The vertical bar graph represents the average score of the students in various domains. It calculates the average using the grouped means method to determine the comparison among the domains. For instance, the ML domain has the best performance benchmark.



Attendance Percentage vs. Score: This is a scatter plot which shows the relationship between two numerical attributes in order to examine the correlation between them. This graph shows that there is clustering where those who have high scores cluster at the high attendance axis.



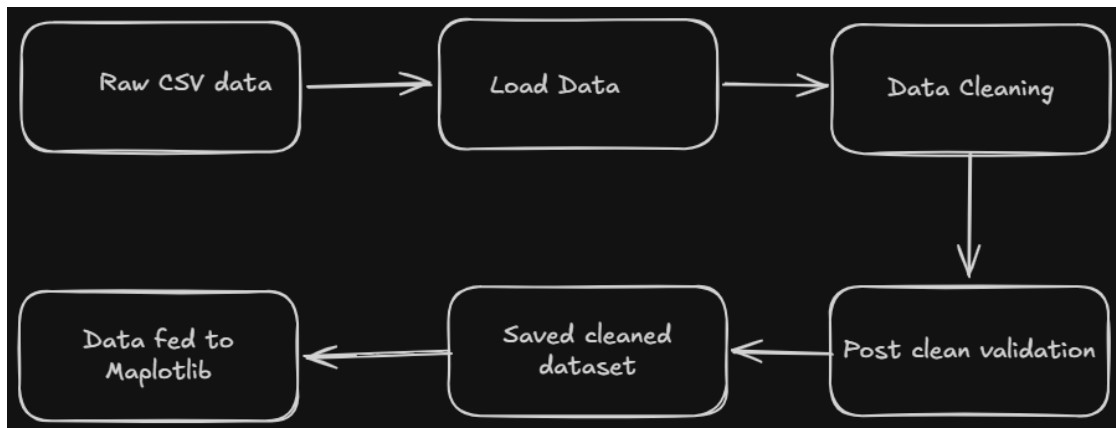
Status of Submission Count: This bar graph gives the count of those who submitted their work versus those who failed to submit. According to the graph, out of 10 students, 7 students submitted successfully while the other 3 failed to submit.



8) Conclusion

Task Number	Task	Takeaways
Task 1	Project initialization	Repository layout creation
Task 2	Virtual environment setup	Setting up venv and installing packages with requirements.txt.
Task 3	CSV Parsing	Comparing manual parsing v/s use of pandas
Task 4	Data cleaning and transformation	Removing duplicate rows, standardizing attribute labels
Task 5	Data Visualization	Plotting labelled charts for better understanding of the cleaned tabular data

Architecture Workflow



9) References

- 1.) https://matplotlib.org/stable/plot_types/index.html
- 2.) <https://www.geeksforgeeks.org/data-analysis/data-cleaning-introduction/>
- 3.) <https://www.digitalocean.com/community/tutorials/parse-csv-files-in-python>